

Shared by KK. This is a copy of a chat between Claude and KK. Content may include unverified or unsafe content that does not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here.

Files hidden when shared

please go through the attached document. i need you to please give me the codes on how to execute this lab. this is the github repo i have to work with:

<https://github.com/SETAPESU26/38-meteor-dodge>

i have already cloned the repo to my system and run the before video. it's stored in a folder called "videos" in the folder in my system. give me the updated codes that i should replace the old codes with to run this lab please

I'll start with the instructions file and the repo.

Ran 2 commands, read a page

Ran 2 commands, read a page

I checked all of this headless before sending it. Lasers destroy meteors, large meteors split into smaller ones, the shield absorbs a hit, and the multiplier reaches x2 after 10 seconds.

I'm splitting the changes across two messages, so this first batch is the three game-object files. Your root main.py stays exactly as it is. All three files go inside the game/ folder.

1. game/laser.py (new file, create it)

```
python
```

```
import pygame
```

```
LASER_SPEED = 10
```

```
LASER_LENGTH = 18
```

```
LASER_WIDTH = 4
```

```
class Laser:
```

```
    def __init__(self, x, y):
```

```
        # (x, y) is the muzzle: the bottom end of the beam
```

```
        self.x = x
```

```
        self.y = y
```

```

@property
def rect(self):
    return pygame.Rect(int(self.x) - LASER_WIDTH // 2, int(self.y) - LASER_LENGTH,
                        LASER_WIDTH, LASER_LENGTH)

def update(self):
    self.y -= LASER_SPEED

def off_screen(self):
    return self.y < 0

def hits(self, meteor):
    # circle (meteor) vs rectangle (laser): find the closest point on the beam
    r = self.rect
    nx = max(r.left, min(meteor.x, r.right))
    ny = max(r.top, min(meteor.y, r.bottom))
    return (meteor.x - nx) ** 2 + (meteor.y - ny) ** 2 < meteor.radius ** 2

def draw(self, screen):
    r = self.rect
    pygame.draw.rect(screen, (90, 255, 140), r, border_radius=2)
    pygame.draw.rect(screen, (230, 255, 235), r.inflate(-2, 0), border_radius=2)

```

2. game/meteor.py (replace the whole file)

```

python
import pygame
import random
import math

```

LARGE_RADIUS = 20 # meteors at or above this radius split when destroyed

class Meteor:

```
def __init__(self, width, x=None, y=None, radius=None, vx=None, vy=None):
```

```
    # With no arguments this spawns a random meteor above the screen.
```

```
    # Fragments created by split() pass their own position/size/velocity.
```

```
    self.x = random.randint(0, width) if x is None else x
```

```
    self.y = -30 if y is None else y
```

```
    self.radius = random.randint(12, 28) if radius is None else radius
```

```
    if vx is None or vy is None:
```

```
        angle = random.uniform(70,110)
```

```
        speed = random.uniform(2,5)
```

```
        vx = math.cos(math.radians(angle))*speed
```

```
        vy = math.sin(math.radians(angle))*speed
```

```
    self.vx = vx
```

```
    self.vy = vy
```

```
    self.color = (
```

```
        random.randint(160,220),
```

```
        random.randint(80,120),
```

```
        random.randint(40,80)
```

```
)
```

```
    self.rot = 0
```

```
    self.rot_speed = random.uniform(-3,3)
```

```
def is_large(self):
```

```
    return self.radius >= LARGE_RADIUS
```

```

def split(self):
    """Large meteors break into 2-3 smaller fragments that fly apart.
    Small meteors return an empty list (they dissolve completely)."""
    if not self.is_large():
        return []
    offsets = random.choice([[-40, 40], [-50, 0, 50]])
    child_radius = max(8, self.radius // 2)
    fragments = []
    for off in offsets:
        a = math.radians(off + random.uniform(-10, 10))
        # rotate the parent's velocity by 'a' so fragments diverge outwards
        vx = self.vx*math.cos(a) - self.vy*math.sin(a)
        vy = self.vx*math.sin(a) + self.vy*math.cos(a)
        boost = random.uniform(1.0, 1.4)
        frag = Meteor(0, x=self.x, y=self.y, radius=child_radius,
                      vx=vx*boost, vy=vy*boost)
        frag.color = self.color
        fragments.append(frag)
    return fragments

def update(self):
    self.x+=self.vx; self.y+=self.vy
    self.rot=(self.rot+self.rot_speed)%360

def off_screen(self, width, height):
    return self.y > height + 60 or self.x < -60 or self.x > width + 60

def collides(self, rect):

```

```

cx,cy=rect.centerx,rect.centery
dx,dy=self.x-cx,self.y-cy
return (dx**2+dy**2)**0.5 < self.radius + 16

```

```

def draw(self, screen):

```

```

    import math

```

```

    pts=[]

```

```

    for i in range(7):

```

```

        angle=math.radians(self.rot+i*(360/7))

```

```

        r=self.radius*(0.8+0.2*(i%2))

```

```

        pts.append((int(self.x+r*math.cos(angle)),int(self.y+r*math.sin(angle))))

```

```

    pygame.draw.polygon(screen,self.color,pts)

```

```

    inner=[(int(self.x+(r*0.5)*math.cos(math.radians(self.rot+i*(360/7)))),

```

```

            int(self.y+(r*0.5)*math.sin(math.radians(self.rot+i*(360/7)))))

```

```

        for i,(cx,cy) in enumerate(pts)]

```

```

    pygame.draw.polygon(screen,tuple(max(0,c-40) for c in self.color),inner)

```

```

class ShieldOrb:

```

```

    """A glowing energy orb that drifts down the screen, swaying side to side."""

```

```

    def __init__(self, width):

```

```

        self.base_x = random.randint(60, width - 60)

```

```

        self.x = self.base_x

```

```

        self.y = -20

```

```

        self.vy = random.uniform(1.2, 2.0)

```

```

        self.t = random.uniform(0, 6.28)

```

```

        self.radius = 13

```

```

def update(self):
    self.y += self.vy
    self.t += 0.05
    self.x = self.base_x + math.sin(self.t) * 40

def off_screen(self, height):
    return self.y > height + 30

def collides(self, rect):
    dx, dy = self.x - rect.centerx, self.y - rect.centery
    return (dx**2 + dy**2) ** 0.5 < self.radius + 16

def draw(self, screen):
    pulse = 1 + 0.15 * math.sin(self.t * 3)
    r = int(self.radius * pulse)
    cx, cy = int(self.x), int(self.y)
    glow = pygame.Surface((r*4, r*4), pygame.SRCALPHA)
    pygame.draw.circle(glow, (80, 220, 255, 60), (r*2, r*2), r*2)
    screen.blit(glow, (cx - r*2, cy - r*2))
    pygame.draw.circle(screen, (80, 220, 255), (cx, cy), r)
    pygame.draw.circle(screen, (220, 250, 255), (cx, cy), max(2, r // 2))

```

3. game/ship.py (replace the whole file)

```

python
import pygame

```

```

SPEED = 5

```

```

class Ship:

```

```

def __init__(self, x, y):
    self.rect = pygame.Rect(x-20, y-20, 40, 40)
    self.color = (80, 160, 240)
    self.trail = []
    self.shield = False # True after collecting a shield orb; absorbs one hit

```

```

def move(self, keys, width, height):
    dx=dy=0
    if keys[pygame.K_LEFT] or keys[pygame.K_a]: dx=-SPEED
    if keys[pygame.K_RIGHT] or keys[pygame.K_d]: dx=SPEED
    if keys[pygame.K_UP] or keys[pygame.K_w]: dy=-SPEED
    if keys[pygame.K_DOWN] or keys[pygame.K_s]: dy=SPEED
    self.rect.x=max(0,min(width-self.rect.width,self.rect.x+dx))
    self.rect.y=max(0,min(height-self.rect.height,self.rect.y+dy))
    self.trail.append(tuple(self.rect.center))
    if len(self.trail)>10: self.trail.pop(0)

```

```

def draw(self, screen):
    for i,pos in enumerate(self.trail):
        alpha=20+i*20
        r=3+i//2
        s=pygame.Surface((r*2,r*2),pygame.SRCALPHA)
        pygame.draw.circle(s,(80,160,240,alpha),(r,r),r)
        screen.blit(s,(pos[0]-r,pos[1]-r))

    # ship body
    cx,cy=self.rect.center
    pts=[(cx,cy-18),(cx-14,cy+14),(cx,cy+6),(cx+14,cy+14)]
    pygame.draw.polygon(screen,self.color,pts)

```

```

# engine glow
pygame.draw.circle(screen,(255,180,60),(cx,cy+12),5)

# shield bubble
if self.shield:
    bubble=pygame.Surface((64,64),pygame.SRCALPHA)
    pygame.draw.circle(bubble,(80,220,255,50),(32,32),30)
    pygame.draw.circle(bubble,(120,235,255,200),(32,32),30,3)
    screen.blit(bubble,(cx-32,cy-32))

```

These three files are new building blocks, and the old game engine doesn't know about them yet. That means python main.py will crash until the engine file arrives in the next message. To check that they're in place, run this from the project root:

```
python -c "import game.laser, game.meteor, game.ship; print('OK')"
```

Let me know when it prints OK and I'll send the remaining file, game_engine.py. It's the one that wires lasers, splitting, shields and the multiplier into the game.

ok, done. it printed ok

Here's the last file, and it wires everything together. It goes at game/game_engine.py. Replace the whole file.

```

python

import pygame
import random

from game.ship import Ship
from game.meteor import Meteor, ShieldOrb
from game.laser import Laser

```

```
WIDTH,HEIGHT=700,520
```

```
FPS=60
```

```
BG=(8,5,20)
```

```
class GameEngine:
```

```

def __init__(self):
    pygame.init()

    self.screen=pygame.display.set_mode((WIDTH,HEIGHT))

    pygame.display.set_caption("Meteor Dodge")

    self.clock=pygame.time.Clock()

    self.font=pygame.font.SysFont("monospace",26,bold=True)

    self.big_font=pygame.font.SysFont("monospace",46,bold=True)

    self.stars=[(random.randint(0,WIDTH),random.randint(0,HEIGHT),random.randint(1,3))
for _ in range(80)]

    self.reset()


def reset(self):

    self.ship=Ship(WIDTH//2,HEIGHT-80)

    self.meteors=[]

    self.lasers=[]

    self.orb=None

    self.orb_timer=random.randint(180,360) # frames until the first orb appears

    self.timer=0

    self.spawn_interval=60

    self.score=0

    self.frames=0    # total frames survived (used for the Time display)

    self.streak=0    # frames survived since the last hit

    self.multiplier=1

    self.game_over=False

    self.started=False


def fire_laser(self):

    # fire from the nose of the ship

```

```
self.lasers.append(Laser(self.ship.rect.centerx,self.ship.rect.centery-18))
```

```
def update_orb(self):
```

```
    if self.orb is None:
```

```
        # only count down to a new orb while the ship has no shield
```

```
        if not self.ship.shield:
```

```
            self.orb_timer-=1
```

```
            if self.orb_timer<=0: self.orb=ShieldOrb(WIDTH)
```

```
        return
```

```
self.orb.update()
```

```
if self.orb.collides(self.ship.rect):
```

```
    self.ship.shield=True
```

```
    self.orb=None
```

```
    self.orb_timer=random.randint(480,900)
```

```
elif self.orb.off_screen(HEIGHT):
```

```
    self.orb=None
```

```
    self.orb_timer=random.randint(480,900)
```

```
def handle_events(self):
```

```
    for event in pygame.event.get():
```

```
        if event.type==pygame.QUIT: return False
```

```
        if event.type==pygame.KEYDOWN and event.key==pygame.K_SPACE:
```

```
            if self.game_over: self.reset()
```

```
            elif not self.started: self.started=True
```

```
            else: self.fire_laser()
```

```
    return True
```

```
def update(self):
```

```

if self.game_over or not self.started: return

keys=pygame.key.get_pressed()

self.ship.move(keys,WIDTH,HEIGHT)

self.timer+=1

if self.timer>=self.spawn_interval:

    self.meteors.append(Meteor(WIDTH))

    self.timer=0

    self.spawn_interval=max(20,self.spawn_interval-0.3)

for l in self.lasers[:]:

    l.update()

    hit=next((m for m in self.meteors if l.hits(m)),None)

    if hit:

        self.meteors.remove(hit)

        self.meteors.extend(hit.split()) # large -> fragments, small -> nothing

        self.lasers.remove(l)

self.lasers=[l for l in self.lasers if not l.off_screen()]

self.update_orb()

for m in self.meteors[:]:

    m.update()

    if m.collides(self.ship.rect):

        if self.ship.shield:

            self.ship.shield=False    # shield absorbs this one hit

            self.meteors.remove(m)

            self.streak=0              # ...but it still counts as a hit

        else:

            self.game_over=True

self.meteors=[m for m in self.meteors if not m.off_screen(WIDTH,HEIGHT)]

self.frames+=1

```

```
self.streak+=1

self.multiplier=1+self.streak//((10*FPS) # +1x for every 10s without a hit

self.score+=self.multiplier
```

```
def draw(self):

    self.screen.fill(BG)

    for sx,sy,sr in self.stars:

        pygame.draw.circle(self.screen,(200,200,220),(sx,sy),sr)

    for m in self.meteors: m.draw(self.screen)

    for l in self.lasers: l.draw(self.screen)

    if self.orb: self.orb.draw(self.screen)

    self.ship.draw(self.screen)

    tm=self.font.render(f"Time: {self.frames//FPS}s",True,(200,200,240))

    self.screen.blit(tm,(10,10))

    mult_color=(255,210,80) if self.multiplier>1 else (200,200,240)

    mu=self.font.render(f"Multiplier: x{self.multiplier}",True,mult_color)

    self.screen.blit(mu,(10,40))

    sc=self.font.render(f"Score: {self.score}",True,(200,200,240))

    self.screen.blit(sc,(WIDTH-sc.get_width()-10,10))

    if self.ship.shield:

        sh=self.font.render("SHIELD",True,(120,235,255))

        self.screen.blit(sh,(WIDTH-sh.get_width()-10,40))

    if not self.started:

        msg=self.font.render("Press SPACE to launch",True,(180,180,240))

        self.screen.blit(msg,(WIDTH//2-msg.get_width()//2,HEIGHT//2))

    if self.game_over:

        ov=pygame.Surface((WIDTH,HEIGHT),pygame.SRCALPHA)

        ov.fill((0,0,0,150))
```

```

self.screen.blit(ov,(0,0))

m=self.big_font.render("DESTROYED!",True,(220,80,60))

s=self.font.render(f"Survived {self.frames//FPS}s | Score {self.score}",True,(200,200,200))

r=self.font.render("SPACE to Restart",True,(200,200,200))

self.screen.blit(m,(WIDTH//2-m.get_width()//2,HEIGHT//2-40))

self.screen.blit(s,(WIDTH//2-s.get_width()//2,HEIGHT//2+20))

self.screen.blit(r,(WIDTH//2-r.get_width()//2,HEIGHT//2+55))

pygame.display.flip()

```

```

def run(self):
    running=True
    while running:
        running=self.handle_events()
        self.update()
        self.draw()
        self.clock.tick(FPS)
    pygame.quit()

```

Then run python main.py and check each task:

- **Task 1 (laser):** Press SPACE to launch, then SPACE again to fire green lasers from the ship's nose.
- **Task 2 (splitting):** Shoot a big meteor (radius 20 or more) and it breaks into 2-3 smaller fragments that fly apart. Small ones just disappear.
- **Task 3 (shield):** A cyan orb drifts down every so often. Collect it and a bubble appears around the ship with "SHIELD" in the HUD. It absorbs one hit.
- **Task 4 (multiplier):** The multiplier goes up by 1 for every 10 seconds without a hit. It shows in gold on the left of the HUD, and the score is on the right.

Let me know when it's running the way you want, and I'll give you the steps to record the after video and push everything to GitHub.

ok done. i recorded the after video and saved it into the same videos folder with before.mp4. now how to push?

First, record that this is a personal push: the repo you cloned belongs to SETAPESU26, so you need to point it at your own GitHub repo. The handout says not to raise a PR to the main repo.

1. Create your own repo

On GitHub, under your personal account, create a new empty repo named meteor-dodge. Don't add a README or .gitignore.

2. Point your local project at it

Open a terminal in the project folder and run this, replacing YOUR-USERNAME:

```
git remote remove origin
```

```
git remote add origin https://github.com/YOUR-USERNAME/meteor-dodge.git
```

```
git remote -v
```

The last command should show only your own URL.

3. Make a separate commit for each task

The handout asks for one commit per task, so commit file by file:

```
git add game/laser.py
```

```
git commit -m "Task 1: Fix laser firing - add Laser class"
```

```
git add game/meteor.py
```

```
git commit -m "Task 2: Add meteor splitting and ShieldOrb class"
```

```
git add game/ship.py
```

```
git commit -m "Task 3: Add shield state and shield bubble to ship"
```

```
git add game/game_engine.py
```

```
git commit -m "Task 4: Add survival multiplier, score, and wire lasers/splitting/shield into engine"
```

```
git add videos/
```

```
git commit -m "Add before and after gameplay videos"
```

The engine changes touch all four tasks, so that wiring lands in the Task 4 commit. Adding files by name also keeps `__pycache__` out of the repo.

If your videos folder sits outside the project folder, copy it in first.

4. Push

```
git branch -M main
```

```
git push -u origin main
```

If GitHub asks you to log in, use a personal access token as the password, since GitHub no longer accepts account passwords for pushes.

5. Add the chat history

Export this conversation as a PDF or doc. Then put it in the repo and push it:

```
git add chat_history.pdf
```

```
git commit -m "Add chat history export"
```

```
git push
```

The handout wants the videos, code and chat export under a Lab-4 folder in your Lab repo. If your Lab repo is separate from this one, create a Lab-4 folder there, copy in the project folder (game/, main.py, videos/, the chat PDF) and run the same add, commit and push steps from that repo. Also grab this chat's share link for the "Chat/LLM used" item in the submission checklist.

no no. i already have a repo and have the folder in that repo called Lab_04. all the files are inside that folder called Lab_04. i have that repo folder in my system. tell me how to push this folder into github please

Open a terminal in your main repo folder, the one that contains Lab_04, not inside Lab_04 itself.

1. Check for a leftover .git folder from the original clone

If you cloned the SETAPESU26 repo straight into Lab_04, it has its own hidden .git folder. That would make Git treat Lab_04 as a repo inside your repo, and your files wouldn't upload properly. Check:

```
ls -a Lab_04
```

On Windows PowerShell, use `dir -Force Lab_04`. If you see .git listed, delete it:

```
rm -rf Lab_04/.git
```

On Windows PowerShell, use `Remove-Item -Recurse -Force Lab_04\.git`. If you see no `.git`, skip this step.

2. Make sure you're on the right repo and up to date

```
git remote -v
```

```
git pull
```

`git remote -v` should show your own GitHub repo URL, not SETAPESU26.

3. Commit each task separately

The handout asks for a separate commit for each task. The paths below assume the files are directly under `Lab_04`. If your project sits one level deeper, such as `Lab_04/38-meteor-dodge/game/...`, add that folder to each path.

```
git add Lab_04/game/laser.py
```

```
git commit -m "Task 1: Fix laser firing - add Laser class"
```

```
git add Lab_04/game/meteor.py
```

```
git commit -m "Task 2: Add meteor splitting and ShieldOrb class"
```

```
git add Lab_04/game/ship.py
```

```
git commit -m "Task 3: Add shield state and shield bubble to ship"
```

```
git add Lab_04/game/game_engine.py
```

```
git commit -m "Task 4: Add survival multiplier, score, and wire lasers/splitting/shield into engine"
```

```
git add Lab_04/videos
```

```
git commit -m "Add before and after gameplay videos"
```

4. Add the chat export, then push

Export this chat as a PDF, save it inside `Lab_04`, and run this, using your real filename:

```
git add Lab_04/chat_history.pdf
```

```
git commit -m "Add chat history export"
```

```
git push
```

If GitHub asks for a password, paste a personal access token instead of your account password.

5. Check it on GitHub

Open your repo in the browser and confirm that Lab_04 shows the code, the videos folder and the chat PDF, and that the commit history shows one commit per task.

If git push gives an error, paste the message here and I'll sort it out.